

Clipboard Translator — Technical Specification

A cross-platform clipboard translation utility distributed as a single binary. Triggered by a global hotkey, it reads the clipboard, prompts the user for a target action via a small native window, sends the text to an LLM API, and writes the result back to the clipboard.

1. Goals & non-goals

Goals

- Single-binary distribution per OS (macOS, Linux, Windows). No runtime dependencies for the end user.
- Sub-second perceived latency for short clipboard snippets (≤ 500 tokens).
- Minimal, keyboard-driven UI — no mouse required.
- Works fully offline for the UI; only the translation call requires network.
- API key stored in OS-native keychain (macOS Keychain, Windows Credential Manager, Linux Secret Service); env var as fallback.
- User-configurable prompt templates — the built-in prompts in §5.3 are defaults that can be overridden via config.
- Encrypted local history of the last N translations, browsable via a hotkey-triggered viewer.
- Glossary support — user-defined terms with mandated translations, injected into prompts to enforce terminology consistency.

Non-goals

- Batch translation or file-based translation workflows (this is a clipboard tool).
 - A polished settings GUI — config lives in a TOML file; history viewer and glossary editor are the only GUI surfaces beyond the prompt window.
 - Browser extension, mobile app, or service-worker variants.
 - Streaming responses (full text is needed before writing to clipboard).
 - Local model support — the architecture allows it (provider abstraction in §5.1) but no first-class setup wizard ships in v1.
-

2. Tech stack decision

Language: Rust

Rust wins this comparison for a clipboard tool specifically:

Criterion	Rust	Go	Zig	C++
Single static binary	✓	✓	✓	⚠️ glibc/libstdc++ pain
Cross-platform clipboard	✓ <code>arboard</code>	⚠️ patchy	✗	⚠️
Cross-platform global hotkey	✓ <code>global-hotkey</code>	⚠️	✗	⚠️
Native GUI options	✓ <code>egui</code> , <code>iced</code> , <code>slint</code>	⚠️ Fyne (heavy)	✗	✓ Qt (license/binary size)
HTTPS + JSON	✓ <code>reqwest</code> + <code>serde</code>	✓	⚠️	⚠️
Mature ecosystem for this niche	✓	⚠️	✗	⚠️

Go is a reasonable second choice but its GUI story is weak for this kind of tiny popup window. Rust's `egui` immediate-mode GUI compiles into the binary, renders identically on all three platforms, and starts in <50ms.

Key crates

- `arboard` — clipboard read/write across macOS/Linux (X11+Wayland)/Windows.
- `global-hotkey` — system-wide hotkey registration on all three platforms.
- `eframe` / `egui` — minimal immediate-mode GUI for the language picker, history viewer, and glossary editor.
- `tray-icon` — menu-bar/system-tray icon (optional, see §6).
- `request` (with `rustls-tls`) — HTTPS without OpenSSL dependency.
- `serde` / `serde_json` — request/response serialization.
- `tokio` — async runtime for the API call.
- `directories` — XDG/AppData/macOS-standard config paths.
- `toml` — config and glossary file parsing.
- `tracing` + `tracing-subscriber` — structured logging.
- `keyring` — cross-platform OS keychain access (Keychain / Credential Manager / Secret Service).
- `rusqlite` (with the `bundled` feature) — embedded SQLite for history storage; bundled feature avoids dynamic linking.
- `chacha20poly1305` + `argon2` — authenticated encryption for the history database, key derived from a machine-bound secret.
- `whatlang` — fast, dependency-free language detection used for glossary scoping and the same-language guarantee in `fix_grammar/rewrite` modes.
- `tera` or `minijinja` — lightweight templating for user-overridable prompt templates with `{{variable}}` substitution. `minijinja` is preferred (smaller, no GIL/Python heritage, pure Rust).
- `zeroize` — wipes sensitive data (API keys, decrypted history, in-flight clipboard text) from memory on drop.

Why this stays a single binary

- `rustls` instead of OpenSSL → no dynamic SSL linking.
- `egui` is pure Rust, no GTK/Qt dependency.
- All assets (icons, fonts) bundled via `include_bytes!`.

Resulting binary size: ~16–24 MB per platform (SQLite + crypto add a few MB; still acceptable for a personal tool).

3. UX flow

[user copies text] → ⌘⇧T (or Ctrl+Shift+T) → prompt window appears



Translate clipboard
Source preview (3 lines):
> Guten Tag, wie geht es...
1) English
2) Deutsch
3) Türkçe
4) Fix grammar
5) Rewrite for clarity
6) Custom prompt...
▶ last used: 2) Deutsch
[Esc to cancel]



user presses 1-6 (or clicks)
or Enter to repeat last



Translating... ▄▄▄▄▄▄▄▄▄▄



clipboard is replaced; window auto-closes;
brief OS notification: "Translation copied"

Interaction details

- **Primary hotkey:** `Cmd+Shift+T` (macOS), `Ctrl+Shift+T` (Linux/Windows). Opens the prompt window. Configurable.
- **History hotkey:** `Cmd+Shift+H` / `Ctrl+Shift+H`. Opens the history viewer (see below). Configurable; can be disabled.
- **Number-key shortcuts:** Pressing `1`–`6` on the prompt window immediately triggers that action — no Enter required. This is the primary interaction mode.
- **Enter:** Repeats the last-used action (see "Last-action recall" below). On first launch with no history, Enter is a no-op.
- **Esc:** Cancels and restores focus to the previous app.
- **Slot 4 — Fix grammar:** Conservative editing. Corrects grammar, spelling, and clear typos only. Does not rephrase, restructure, or change word choice. Stays in the source language.
- **Slot 5 — Rewrite for clarity:** Aggressive editing. Rewrites for clarity, flow, and concision. May restructure sentences and change phrasing. Preserves meaning and tone but not exact wording. Stays in the source language.
- **Slot 6 — Custom prompt:** Opens a text input where the user types an instruction (e.g. "translate to formal Spanish" or "make this sound more diplomatic"). The clipboard text is the input; the typed instruction becomes part of the system prompt.
- **Source preview:** Shows first ~100 chars of clipboard so the user can confirm they copied the right thing. Truncated with `...` if longer.
- **Empty / non-text clipboard:** Prompt window shows error state ("Clipboard is empty or not text"), Esc to dismiss.

Last-action recall

When `[ui] remember_last_action = true` (default), the prompt window:

- Highlights the last successfully-used action with a `▶` indicator and "last used:" label below the menu.
- Maps `Enter` to that action — pressing Enter is equivalent to pressing the corresponding number key.
- Persists the last action across app restarts in a tiny state file (`<config_dir>/state.toml`). Custom prompts are NOT persisted — slot 6 always reopens the text input rather than re-running the last typed instruction (which would feel surprising).
- Per-language-pair memory is out of scope; one global "last action" is enough.

History viewer

A second window, summoned by the history hotkey or via the tray icon:

History[search: ____]

▶ 2 min ago

DE → EN

"Guten Tag, wie..."

▶ 14 min ago

rewrite

"The api need to..."

▶ 32 min ago

grammar

"He dont know that..."

▶ 1 h ago

EN → TR

"Please review the..."

▶ 3 h ago

custom

"Make this formal..."

[Enter] copy result

[s] copy source

[d] delete

[Shift+Del] clear all

[Esc] close

- Arrow keys to navigate, Enter to copy the result back to clipboard, **s** to copy the original source instead.
- Type-to-search filters in real time across both source and result text.
- **d** deletes the highlighted entry; **Shift+Del** clears all (with confirmation modal).
- History is capped at N entries (default 100, configurable). Oldest entries are dropped when the cap is exceeded.
- Each entry shows: relative timestamp, action (lang pair / **grammar** / **rewrite** / **custom**), and a truncated source preview.

Glossary editor

Glossary entries are managed by hand-editing the glossary TOML file (see §6). The app provides one convenience: a tray icon menu item "Open glossary" that launches the file in the system default editor. No in-app editor in v1 — the file format is simple enough that a text editor is the right tool, and this keeps the UI surface minimal.

Window behavior

- Prompt and history windows appear centered on the active monitor; prompt is ~480×360 px, history is ~640×480 px.
- Always-on-top, no decorations, minimal chrome.
- Auto-focuses on appearance so keyboard shortcuts work immediately.
- Prompt window auto-closes on success or Esc; history window stays open until Esc.

4. Architecture

main.rs (entry point)

- loads config, glossary, prompt templates
- resolves API key (keychain → env var → setup wizard)
- opens encrypted history database
- registers global hotkeys (prompt, history)
- spawns tray icon (optional)
- enters event loop

on prompt hotkey:

prompt_window

- reads clipboard via arboard
- displays egui modal
- returns Action enum:
Translate(Lang) | FixGrammar | Rewrite | Custom(String)

translator

- selects prompt template for chosen Action
- looks up applicable glossary entries by language pair
- renders template with glossary block injected
- calls LLM API (async, with 30s timeout)
- returns Result<String, TranslateError>

output_handler

- writes result to clipboard
- records entry to encrypted history DB
- shows OS notification
- logs metadata only (text NOT logged)

on history hotkey:

history_window

- queries encrypted SQLite store
- displays scrollable list with search
- allows copy-back / delete / clear

```

src/
├─ main.rs           # entry, event loop, hotkey wiring
├─ config.rs         # TOML config load/save, defaults, schema validation
├─ clipboard.rs      # arboard wrapper, text-only filtering
├─ secrets.rs        # keychain read/write with env var fallback
├─ ui/
│   ├─ mod.rs
│   ├─ prompt.rs     # main language picker window
│   ├─ custom_prompt.rs # text input for option 5
│   ├─ history.rs    # history viewer window
│   └─ setup.rs      # first-launch API key setup wizard
├─ llm/
│   ├─ mod.rs
│   ├─ client.rs     # HTTP client, retry logic
│   ├─ anthropic.rs  # Claude provider impl
│   ├─ openai.rs     # OpenAI-compatible provider impl
│   ├─ prompts.rs    # built-in default templates (const &str)
│   └─ templates.rs  # user-override loader, minijinja rendering
├─ glossary.rs       # glossary file load, language-pair scoping, prompt-block
├─ formatting
├─ history/
│   ├─ mod.rs
│   └─ store.rs      # encrypted SQLite operations
│       └─ crypto.rs  # ChaCha20-Poly1305 + Argon2 key derivation
├─ notify.rs         # cross-platform OS notifications
└─ error.rs          # unified error type

```

5. LLM integration

5.1 Provider abstraction

```

rust

#[async_trait]
trait LlmProvider {
    async fn complete(
        &self,
        system: &str,
        user: &str,
    ) -> Result<String, TranslateError>;
}

```

Two implementations ship:

- **AnthropicProvider** — Claude Haiku 4.5 / Sonnet 4.6
- **OpenAiCompatibleProvider** — works with OpenAI, Gemini (via OpenAI-compatible endpoint), DeepSeek, local Ollama

This abstraction means swapping providers is a config change, not a code change.

5.2 Recommended default: Claude Haiku 4.5

Rationale (from the prior conversation context, restated for completeness):

- Latency: typically 300–600ms for short snippets — feels instant.
- Instruction-following: returns bare translations without preamble more reliably than Flash-tier models.
- Cost at personal scale: ~\$0.20/month at heavy use (100 translations/day).
- Quality on DE/EN/TR is strong; Turkish morphology handled well.

Config allows override to:

- `gemini-2.5-flash` — cheaper, slightly lower instruction adherence.
- `claude-sonnet-4-6` — better for "rewrite for clarity" on long-form content where structural changes matter; ~3x cost.
- Local model via Ollama — for offline/privacy use, expect lower quality.

5.3 Prompt templates

Templates are rendered with `minijinja` and use `{{variable}}` syntax. The app ships built-in defaults as `const &str` in `src/llm/prompts.rs`. If the user provides override templates in their config directory (`templates/translate.j2`, `templates/fix_grammar.j2`, `templates/rewrite.j2`, `templates/custom.j2`), those are loaded at startup and used in place of the defaults.

Available variables in all templates:

- `{{ source_language }}` — detected via `whatlang` (best-effort; may be `unknown`).
- `{{ glossary_block }}` — pre-formatted glossary terms relevant to the language pair, or empty string if none apply (see §5.4).

Translate-specific variables:

- `{{ target_language }}` — human-readable name, e.g. `"German"` or `"Türkçe"`.

Custom-specific variables:

- `{{ user_instruction }}` — the text the user typed in custom mode.

Built-in translate template:

You are a translation engine. Translate the user's text into `{{ target_language }}`.

Rules:

- Output ONLY the translation. No preamble, no quotes, no explanation, no notes.
 - Preserve formatting: line breaks, lists, code blocks, markdown.
 - Preserve proper nouns, code, URLs, and technical terms.
 - Match the register (formal/informal) of the source.
 - If the text is already in `{{ target_language }}`, return it unchanged.
- `{{ glossary_block }}`

Built-in fix_grammar template (conservative):

You are a copy editor performing a grammar pass. Fix grammar, spelling, and punctuation errors in the user's text.

Rules:

- Detect the source language and respond IN THE SAME LANGUAGE. Do NOT translate.
 - Output ONLY the corrected text. No preamble, no quotes, no explanation.
 - Make the MINIMUM changes needed to fix actual errors.
 - Do NOT rephrase, restructure, or substitute words for stylistic reasons.
 - Do NOT change vocabulary, tone, register, or sentence structure.
 - Do NOT "improve" awkward but grammatically valid phrasing.
 - Preserve formatting exactly: line breaks, lists, code blocks, markdown.
 - If the text has no errors, return it unchanged.
- `{{ glossary_block }}`

Built-in rewrite template (aggressive):

You are a writing editor. Rewrite the user's text for clarity, flow, and concision.

Rules:

- Detect the source language and respond IN THE SAME LANGUAGE. Do NOT translate.
 - Output ONLY the rewritten text. No preamble, no quotes, no explanation.
 - Preserve the author's meaning, intent, and overall tone.
 - You MAY restructure sentences, change phrasing, and reorder ideas for clarity.
 - You MAY shorten verbose passages and split run-on sentences.
 - Do NOT add new information or change the substance of what is being said.
 - Preserve formatting: line breaks, lists, code blocks, markdown.
- `{{ glossary_block }}`

Built-in custom template:

You are a text-processing assistant. Apply the following instruction to the user's text:

```
{{ user_instruction }}
```

Rules:

- Output ONLY the result. No preamble, no quotes, no explanation.
- Preserve formatting unless the instruction says otherwise.

```
{{ glossary_block }}
```

Template validation runs at startup: missing files fall back to defaults (with a warning log); malformed templates abort startup with a clear error pointing to the file and line. The `{{ glossary_block }}` variable is optional in user templates — if omitted, glossary support silently degrades for that template.

5.4 Glossary

The glossary is a TOML file at `<config_dir>/glossary.toml`. Each entry maps a source term to a fixed translation, optionally scoped to specific language pairs.

```
toml

[[entry]]
source = "Smart Table"
target = "Smart Table"           # leave product name unchanged
languages = ["*"]                # applies to any pair

[[entry]]
source = "Vorgang"
target = "case"
languages = ["de->en"]

[[entry]]
source = "case"
target = "Vorgang"
languages = ["en->de"]

[[entry]]
source = "GIP"
target = "GIP"
languages = ["*"]                # company name, never translate
note = "Always preserve as-is"
```

At translation time, the translator module:

1. Detects source language (via `whatlang`) and combines with target to form a pair key like `"de->en"`.
2. Filters glossary entries to those matching `"*"` or the current pair.
3. Of those, only includes entries whose `source` term appears in the clipboard text. The matching strategy depends on `[glossary] matching` (see §6):
 - `"word_boundary"` — wraps the term in regex `\b...\b`. Avoids spurious matches like "case" inside "casein". Works for languages with whitespace word separation (English, German, Turkish, French, Spanish, etc.).
 - `"substring"` — plain case-insensitive contains check. Works for any language but produces false positives. Required for CJK languages, Thai, and other no-whitespace scripts.
 - `"auto"` (default) — picks per-translation based on detected source language. Uses `word_boundary` for languages with word separators; falls back to `substring` for `(zho, jpn, tha, lao, mya, khm)`. If language detection returns `unknown`, defaults to `word_boundary` (the safer choice for the target language set).
4. If the filtered set is non-empty, renders this block and substitutes it into `{{ glossary_block }}`:

GLOSSARY — these terms MUST be translated exactly as specified:

- "Smart Table" → "Smart Table"
- "Vorgang" → "case"
- "GIP" → "GIP" (Always preserve as-is)

If the filtered set is empty, `{{ glossary_block }}` resolves to an empty string and the template renders cleanly without trailing whitespace.

Glossary is loaded once at startup and on SIGHUP (Linux/macOS) or via a tray-icon menu item "Reload glossary". This avoids hitting the filesystem on every translation.

5.5 Request shape (Anthropic)

```
json
{
  "model": "claude-haiku-4-5-20251001",
  "max_tokens": 4096,
  "system": "<template above>",
  "messages": [{ "role": "user", "content": "<clipboard text>" }]
}
```

- `max_tokens: 4096` covers ~3000 words of output, enough for any clipboard-sized task.
- No streaming — the full string is needed before writing to clipboard.
- 30-second timeout; on timeout, show error and leave clipboard untouched.

5.6 Post-processing

Even with strict prompts, models occasionally wrap responses in quotes or add a trailing newline. Sanitize before writing to clipboard:

1. Trim leading/trailing whitespace.
 2. If the entire response is wrapped in matching `"..."` or `“...”` or `„...”` quotes AND the source wasn't, strip the wrapping quotes.
 3. Strip a leading `"Here is the translation:"` / `"Translation:"` / `"Übersetzung:"` pattern (regex fallback for prompt failures).
 4. Preserve all internal formatting.
-

6. Configuration

File locations

Per-OS config directory (`<config_dir>` below):

- macOS: `~/Library/Application Support/clipboard-translator/`
- Linux: `~/.config/clipboard-translator/`
- Windows: `%APPDATA%\clipboard-translator\`

Files within `<config_dir>`:

- `config.toml` — main settings
- `state.toml` — runtime state (last-used action, etc.); auto-managed, not for hand-editing
- `glossary.toml` — glossary entries (optional, see §5.4)
- `templates/translate.j2`, `templates/fix_grammar.j2`, `templates/rewrite.j2`, `templates/custom.j2` — optional template overrides
- `history.db` — encrypted SQLite database for translation history

Schema

```

toml

[provider]
# "anthropic" | "openai" | "gemini" | "ollama"
type = "anthropic"
model = "claude-haiku-4-5-20251001"
base_url = "https://api.anthropic.com/v1"    # override for proxies / Ollama
timeout_seconds = 30

[provider.api_key]
# Resolution order: keychain → env_var → setup wizard
# "service" and "account" identify the keychain entry
source = "keychain"           # "keychain" | "env" | "prompt"
service = "clipboard-translator"
account = "anthropic"
env_var = "ANTHROPIC_API_KEY"  # used if source = "env" or as fallback

[hotkey]
# Modifier names: "cmd", "ctrl", "alt", "shift"
# "cmd" maps to Ctrl on Linux/Windows automatically
prompt = { modifiers = ["cmd", "shift"], key = "T" }
history = { modifiers = ["cmd", "shift"], key = "H" }  # set to null to disable

[ui]
show_preview = true
auto_close_seconds = 30
notify_on_success = true
remember_last_action = true    # pre-select last action when opening prompt
confirm_size_threshold = 2000  # chars; 0 = never confirm

[tray]
# The tray icon is the discovery surface for glossary, history, and settings.
# Default-on. Users can hide it via right-click → "Hide icon" – that hide
# state persists here.
visible = true
# Shortcut shown in the "Hide icon" confirmation modal so users know how
# to reach the app without the tray. Mirrors [hotkey.prompt] for clarity.
# Read-only mirror – actual hotkey is set in [hotkey.prompt].

[languages]
# Slots 1-3 are user-configurable target languages
# Slot 4 = "Fix grammar" (conservative), slot 5 = "Rewrite for clarity" (aggressive)
# Slot 6 = "Custom prompt"
slot_1 = { label = "English", code = "en" }
slot_2 = { label = "Deutsch", code = "de" }
slot_3 = { label = "Türkçe", code = "tr" }

[templates]
# Optional. If a path is set and the file exists, it overrides the built-in template
# Path must be absolute or relative to the app's root directory.

```

```

# Paths are relative to <config_dir>. Set to null or omit to use built-ins.
translate = "templates/translate.j2"
fix_grammar = "templates/fix_grammar.j2"
rewrite = "templates/rewrite.j2"
custom = "templates/custom.j2"

[glossary]
enabled = true
file = "glossary.toml"           # relative to <config_dir>
# Whether glossary matches are case-sensitive when scanning the source text
case_sensitive = false
# How to detect a glossary term in the source text:
#   "auto"           - word-boundary for whitespace-using languages, substring for CJK/Thai
#   "word_boundary" - always use \b regex (will miss matches in CJK/Thai)
#   "substring"     - always use plain contains (may produce false positives like "c
matching = "auto"

[history]
enabled = true
# Max entries retained. At 100, full-text search across encrypted blobs costs
# <10ms per query (every row is decrypted on search). Raising this significantly
# trades search latency for retention; no FTS index is built.
max_entries = 100
# When false, history records action + language pair + timestamp + char count only
# (no source or result text). Useful for high-sensitivity environments.
store_text = true
# Confirmation prompt before "Clear all"
confirm_clear = true

[logging]
level = "info"                   # "error" | "warn" | "info" | "debug"
file = null                      # null = no file logging
# IMPORTANT: clipboard text is NEVER logged regardless of level

```

API key resolution

Resolution order on startup, with fallback on miss:

1. **Keychain** (`source = "keychain"`): read from OS keychain using `service` + `account`.
Uses macOS Keychain via `Security.framework`, Windows Credential Manager via `wincred`, Linux Secret Service via D-Bus (requires `gnome-keyring`, `kwallet`, or equivalent running).
2. **Environment variable** (`source = "env"` or keychain miss): read `env_var`.
3. **Setup wizard** (no key found anywhere): see below.

The keychain entry is written with appropriate ACLs:

- macOS: bound to the application bundle identifier; other apps cannot read it without explicit user consent.
- Windows: scoped to the current user.
- Linux: stored in the default Secret Service collection.

If the keychain is unavailable (e.g. headless Linux without a Secret Service daemon), the app logs a warning at startup and falls back to env var. The setup wizard will not offer keychain storage in this case.

Setup wizard

Triggered on first launch when no API key can be resolved. A single window with these steps:

Step 1 — Provider selection. Dropdown of supported providers (Anthropic, OpenAI, Gemini, Ollama). Prefills based on `[provider] type` in config if set; otherwise defaults to Anthropic. Below the dropdown, a link: "Get your API key →" opens the provider's key-management page in the default browser.

Step 2 — Key entry. Password-style input field (text masked by default with a show/hide toggle). Below it, the storage choice:

- ☒ *Keychain* (recommended) — disabled and explained if keychain is unavailable
- ☐ *Environment variable* — shows the env var name from config

Step 3 — Verification. Two sequential checks, both visible to the user with their own status indicator:

1. **Connectivity check** (free, ~100ms): calls the provider's lightweight endpoint — `GET /v1/models` for OpenAI/Gemini/Ollama, or for Anthropic, a minimal `POST /v1/messages` with `max_tokens: 1` since Anthropic has no free auth-check endpoint. This catches the most common failures: typo'd key, expired key, wrong provider selected, no network. On failure, shows a specific error (`401 Invalid API key`, `403 Permission denied`, `Network unreachable`, etc.) and lets the user fix and retry without re-entering the key.
2. **Sample translation** (optional, ~\$0.0001 in tokens): a checkbox "Test with a real translation (recommended)", default checked. Translates the string `"Hello, world."` to German using the configured model. Shows source and result side-by-side. This catches more subtle failures: model name typo, account billing not set up, model not available on the user's tier, prompt format incompatibility for non-Anthropic providers via the OpenAI-compatible path.

Only after both checks pass does the wizard offer the "Save and start" button, which writes the key to the chosen storage and launches the main event loop. If only check 1 passes and the user unchecked check 2, the wizard warns: "Skipping the test translation. The app will save your key, but you may hit errors on first real use."

Failure recovery. If either check fails, the wizard stays open with the error displayed. The user can edit the key, switch providers, or change the model in `config.toml` (a "Open config" button is shown for this) without losing their typed key. Esc cancels the wizard and exits the app — better than starting up in a half-configured state.

7. Cross-platform concerns

Global hotkey

- macOS requires Accessibility permission. App must detect missing permission on first launch and open System Settings → Privacy & Security → Accessibility with a clear instruction modal.
- Linux: works out of the box on X11; on Wayland, global hotkeys require a portal (xdg-desktop-portal-gnome ≥ 46 or KDE equivalent). Document this limitation.
- Windows: registers via `RegisterHotKey` Win32 API, no special permissions.

Clipboard

- macOS and Windows: text-only filter is straightforward.
- Linux X11: read both `CLIPBOARD` and `PRIMARY` selections; prefer `CLIPBOARD`.
- Linux Wayland: clipboard works via `wl-clipboard` protocol; `arboard` handles this transparently but document that the user may need `wl-clipboard` installed on minimal distros.

Keychain

- macOS: Keychain Services via the `keyring` crate (uses `Security.framework`). Item is scoped to the app's bundle identifier; reading from another app triggers an OS prompt.
- Windows: Credential Manager via `wincred`. Items are user-scoped.
- Linux: Secret Service over D-Bus. Requires a running secret service provider (`gnome-keyring-daemon`, `kwallet`, or `keepassxc` with Secret Service enabled). On headless servers or minimal distros without one, the app falls back to env var and logs a warning at startup.

History storage

- SQLite database at `<config_dir>/history.db`, accessed via `rusqlite` with the `bundled` feature (no system SQLite dependency).
- All source and result text columns are encrypted at the application layer with ChaCha20-Poly1305. The encryption key is derived via Argon2 from a per-install random secret stored in the same OS keychain entry namespace as the API key (separate account name: `history-key`). On first run, the secret is generated and stored.
- Metadata columns (timestamp, action, language pair, char count) are stored in plaintext to enable search and filtering without decrypting every row.
- Schema:

```
sql

CREATE TABLE entries (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  created_at INTEGER NOT NULL,           -- unix epoch
  action TEXT NOT NULL,                 -- 'translate' | 'fix_grammar' | 'rewrite'
  source_lang TEXT,                     -- ISO 639-1, may be NULL
  target_lang TEXT,                     -- ISO 639-1, NULL for fix_grammar/rewrite
  char_count INTEGER NOT NULL,
  source_ciphertext BLOB,               -- NULL if store_text = false
  source_nonce BLOB,
  result_ciphertext BLOB,               -- NULL if store_text = false
  result_nonce BLOB
);
CREATE INDEX idx_created_at ON entries (created_at DESC);
```

- If the keychain is unavailable and history is enabled, the encryption key falls back to a key file at `<config_dir>/history-key` with `0600` permissions. Documented as less secure and surfaced as a startup warning.
- "Clear all" deletes all rows but leaves the encryption key in place. A separate "Reset history" action (only via direct file deletion, not in-app) wipes both.

Notifications

- macOS: `osascript` fallback or `notify-rust` crate (uses `NSUserNotification`).
- Linux: D-Bus via `notify-rust`.
- Windows: Toast notifications via `notify-rust`.

Tray icon

Default-on. The tray icon is the discovery surface for the app's non-hotkey features (history viewer, glossary file, reload, quit).

Menu (left-click on macOS, right-click on Linux/Windows):

- *Translate clipboard* — equivalent to pressing the prompt hotkey
- *Open history* — equivalent to the history hotkey
- *Open glossary* — opens `glossary.toml` in the system default editor
- *Reload glossary* — re-reads the glossary file without restart
- —
- *Hide icon* — hides the tray icon and persists `[tray] visible = false`. Before hiding, shows a confirmation modal: "Hiding the tray icon. You can still summon the app with `Cmd+Shift+T`. To show the icon again, edit `<config_dir>/config.toml` and set `[tray] visible = true`, or run the binary with `--show-tray`." The shortcut shown in the modal reads from the actual configured hotkey, not a hardcoded string.
- *Quit*

Re-showing after hide: Two paths, both documented in the hide modal:

1. Edit `config.toml` and set `visible = true`, then restart.
2. Run the binary with the `--show-tray` flag, which sets `visible = true` in config and starts normally.

Per-OS notes:

- macOS: shows in menu bar; menu opens on click.
- Linux: requires StatusNotifierItem support (most modern DEs have it). On DEs without it, log a warning and run headless — hotkey still works.
- Windows: standard system tray with right-click menu.

App bundle / packaging

- macOS: `cargo-bundle` produces `.app` bundle; signing + notarization required for Gatekeeper, but for personal use users can right-click → Open. For wider distribution, ad-hoc signing instructions in README.
- Linux: ship a single binary plus a `.desktop` file and an icon. AppImage as an optional alternative.
- Windows: ship the `.exe` directly; optionally produce an MSI via `cargo-wix` for a polished install.

8. Error handling

Error	User-facing behavior
Empty clipboard	Prompt shows: "Clipboard is empty. Copy some text first." Esc to dismiss.
Non-text clipboard (image, file)	Same as empty.
Missing API key	Setup wizard window with instructions; does not proceed to translation.
Keychain unavailable on startup	Warning logged; falls back to env var. Setup wizard hides keychain option.
Keychain read fails after previous success	Toast: "Keychain access denied. Translation aborted." Possibly OS-level prompt the user dismissed.
Network error	Toast notification: "Translation failed: network error. Clipboard unchanged."
API error (4xx)	Toast: "Translation failed: <reason>. Clipboard unchanged."
API error (5xx)	One automatic retry with exponential backoff (1s, 2s); then error toast.
Timeout	Toast: "Translation timed out. Clipboard unchanged."
Hotkey already in use	Startup error logged; tray icon (if enabled) shows warning state.
Rate limit (429)	Toast: "Rate limited. Try again in a moment." Respect <code>Retry-After</code> header.
Glossary file malformed	Startup warning; glossary disabled for this session; tray icon shows warning. App still translates without glossary.
Template file malformed	Startup error with file + line; aborts startup. (Strict: bad templates would silently break every translation.)
Template references unknown variable	Startup error with file + line; aborts startup.
History DB corruption	Toast on startup: "History database unreadable; new history will not be saved." App continues to function; user can delete <code>history.db</code> manually to reset.
History DB write fails mid-session	Logged; translation succeeds and clipboard is updated; only the history record is lost. Toast suppressed (clipboard update is the user's primary outcome).
History encryption key missing	Treat as corruption case above.

Critical invariant: On any failure, the original clipboard contents are preserved. The clipboard is overwritten only after a successful API response and post-processing. History writes are best-effort and never block clipboard updates.

9. Privacy & security

- **Clipboard text is never written to logs** — not in info, not in debug, not in crash reports. Only metadata is logged: timestamp, character count, action chosen, success/failure, latency.
- **API keys** live in the OS keychain (preferred) or environment variable. They are never written to the config file in plaintext, never logged, and never included in error messages.
- **History encryption** — source and result text are encrypted at the application layer with ChaCha20-Poly1305 (AEAD). The key is derived via Argon2 from a 32-byte secret stored alongside the API key in the OS keychain. Each row uses a fresh nonce. Metadata (timestamps, language pairs, char counts) is plaintext for searchability.
- **History opt-out** — `[history] enabled = false` disables the database entirely. `store_text = false` retains metadata but skips storing source/result text. Useful for high-sensitivity work.
- **Glossary file** is plaintext TOML. Sensitive terminology should not be put in the glossary; it's not a secret store.
- **TLS** via `rustls` with system root certs. No fallback to HTTP.
- **No telemetry. No analytics. No remote update check.**
- **Size confirmation** — the `confirm_size_threshold` config option requires explicit confirmation before sending clipboards larger than N characters to the API. Default 2000. Prevents accidentally sending large pastes to the API.
- **Memory hygiene** — the API key, decrypted history entries, and clipboard text in transit through the translator are stored in `zeroize::Zeroizing<String>` wrappers where practical, so they are wiped from memory when dropped rather than lingering in freed allocations.

10. Build & distribution

Build matrix

- `x86_64-apple-darwin`, `aarch64-apple-darwin` (universal binary via `lipo`)
- `x86_64-unknown-linux-gnu`, `aarch64-unknown-linux-gnu`
- `x86_64-pc-windows-msvc`

CI

- GitHub Actions matrix builds all five targets on push to `main`.
- Releases triggered by git tag `v*. *.*`; artifacts attached to the GitHub Release.
- macOS builds: ad-hoc signed by default; notarization optional via secrets.

Versioning

- Semver. v0.x while iterating; v1.0 once config schema is stable.
-

11. Testing strategy

- **Unit tests** for:
 - Prompt template rendering with and without glossary blocks
 - Response post-processing (quote stripping, preamble removal)
 - Config and glossary parsing, including malformed inputs
 - Glossary scoping logic — language-pair filtering, substring vs word-boundary matching, case sensitivity
 - Glossary `(auto)` matching strategy: verifies word-boundary used for de/en/tr/fr/es, substring used for zho/jpn/tha
 - History encryption round-trips (encrypt → store → load → decrypt)
 - Argon2 key derivation determinism for a given secret
- **Integration tests** for:
 - LLM client using a mocked HTTP server (`(wiremock)`)
 - Keychain read/write on each OS (gated by CI runner type)
 - SQLite history operations end-to-end with an in-memory DB
- **Manual test matrix** documented in `(TESTING.md)`:
 - Each of DE/EN/TR as source for each as target = 9 combinations
 - "Fix grammar" on each language with intentionally introduced typos and grammar errors — verify minimal changes only
 - "Rewrite for clarity" on each language with verbose/run-on input — verify meaning preserved, structure changed
 - Verify "fix grammar" and "rewrite" stay in source language and never translate (the most common failure mode)
 - Custom prompt with a few representative instructions
 - Glossary with `(*)` scope and pair-specific scope, both hit and miss cases
 - Template overrides: valid override, malformed file, missing variable
 - History viewer: search, copy-back, delete, clear all, cap enforcement at `max_entries`
 - Edge cases: empty clipboard, image clipboard, very long text (>10k chars), text with markdown, text with code blocks, text containing the size-confirmation threshold boundary
 - Keychain: first-run setup, key rotation, missing-keychain fallback, denied access
 - Setup wizard: invalid key (401), expired key, wrong provider, no network, model name typo (caught by sample translation but not connectivity check), unchecked sample translation flow, "Open config" recovery path, Esc cancellation
 - Last-action recall: Enter pre-selects correct action; survives restart; custom mode does not auto-replay
 - Tray icon: hide flow, `(--show-tray)` re-enables, manual config edit re-enables, hide persists across restart

- **Latency benchmark:** automated script that measures end-to-end time from hotkey to clipboard update across 20 representative snippets. Target: p50 < 800ms, p95 < 2000ms with Haiku 4.5. Glossary lookup and history write must each add < 5ms.
 - **Fuzzing:** `cargo-fuzz` targets for the glossary parser, template renderer, and history decryption (must reject tampered ciphertext).
-

12. Roadmap beyond v1

The provider abstraction in §5.1 already accepts an OpenAI-compatible base URL, which means Ollama can be configured manually today. What's deferred to v2:

- **v2.0: First-class local model support.** A setup wizard that auto-detects a running Ollama instance, lists available models, downloads one if none are present, and configures the provider settings. Plus prompt-template tuning specifically for smaller local models, which often need more explicit "no preamble" reinforcement than frontier models.
-

13. Open questions

All v1 design questions have been resolved. Remaining decisions are implementation-level and best made during the build:

- Exact `whatlang` confidence threshold below which to treat detection as `unknown` and apply the conservative defaults.
- Whether to bundle a small set of opinionated example glossary entries in the install (probably no — keeps the default state truly empty).
- How aggressively to retry the setup wizard's connectivity check on transient network errors (probably one auto-retry, then surface the error and let the user retry manually).